

# Event Location Resolution Notes

This is a **pre-schema implementation note** for the OWeek demo. It documents the location-resolution flow before the final database structure is locked.

## Purpose

The app needs a reliable chain from an event's venue reference to a parent map POI.

That chain is:

```
event.venue_id -> venues.json -> poi_id -> pois.geojson feature
```

This lets the app:

- show the right display label on event cards
- show the correct parent building or landmark in the POI detail sheet
- center or highlight the correct map pin
- preserve subvenue context without giving every room/lounge its own pin

## Current model idea

For the demo, the data is split conceptually into:

- `venues.json`  
Contains parent venues and subvenues, plus aliases and matching metadata.
- `pois.geojson`  
Contains only top-level parent POIs with real coordinates for map pins.
- `resolveVenue.js`  
Contains the lookup utility that resolves an event venue to its parent POI.

This is **not** the final database design. It is just the current demo architecture.

## Boot-time setup

```
import venuesData from './venues.json';
import poisGeojson from './pois.geojson';
import { createVenueResolver } from './resolveVenue.js';

const resolver = createVenueResolver(venuesData, poisGeojson);
```

## Per-event resolution

```
const resolved = resolver.resolve(event.venue_id);
```

## Intended return shape

The resolver should return enough information for multiple UI surfaces.

```
resolved.displayLabel
// Example: "Room 56, UCC"

resolved.locationHint
// Example: "Room 56"

resolved.poi.properties
// Example: { id: "ucc", name: "University Community Centre", ... }

resolved.poiId
// Example: "ucc"

resolved.coordinates
// Example: [-81.2762, 43.0088]

resolved.fallbackReason
// null if clean match, string if matched via alias/raw label/fallback path

resolved.found
// false if nothing matched
```

## UI uses

### Event card

Use the resolved venue data for clear user-facing location display.

Examples:

- `displayLabel`
- `locationHint`

## POI detail sheet

Use the resolved parent POI to show:

- building/landmark name
- all events tied to that parent POI
- original subvenue name where relevant

## Map centering / pin highlight

Use:

- poiId
- coordinates

This ensures subvenues like UCC 56 or Perth Fireside Lounge still highlight the parent POI pin.

## Debug / admin

Use:

- found
- fallbackReason

This helps catch bad schedule strings, alias mismatches, or missing venue definitions.

## Important modeling decisions

### Parent POIs vs subvenues

Only top-level parent POIs should become map pins.

Examples:

- UCC -> parent POI pin
- UCC 56 -> subvenue inside UCC
- The Spoke -> resolve to UCC pin for the demo
- Perth Fireside Lounge -> resolve to Perth Hall pin

### Audience tags are event metadata

Tags like these are **not venue metadata**:

- ALL
- SCI
- KINGS
- PERTH

- SAUG
- OC

These belong on the event model, not the POI model.

## Informal/fuzzy locations

For the demo, informal or difficult spaces can be skipped or attached to a parent POI later.

## Examples of expected resolution

- UCC 56 -> parent POI ucc
- UCC Atrium -> parent POI ucc
- The Spoke -> parent POI ucc
- The Wave -> parent POI ucc
- Perth Fireside Lounge -> parent POI perth
- Great Hall, Somerville House -> parent POI somerville\_house
- Thames Hall Atrium -> parent POI thames\_hall

## Why this note exists

The goal is to keep the demo architecture clear before the final schema is designed.

This note should act as:

- a reference for implementation
- a prompt anchor for Claude
- a reminder of how events, venues, and POIs connect

## Later migration path

If the project moves beyond static JSON into a real database, this same resolution chain can stay conceptually the same:

```
events table -> venues table -> poi_id -> pois/locations table
```

So this note is useful even before the final database structure exists.